

CSCE 627 THEORY OF COMPUTABILITY · MODULE 06 · LECTURE NOTES

Reductions

One hard problem becomes many.

MODULE OUTCOMES

- Define a mapping reduction and state what it preserves.
- Get the direction right, which is where attempts fail.
- Construct reductions proving several problems undecidable.
- State and apply Rice's theorem.
- Explain the relationship to NP-hardness reductions.

1 The technique

A REDUCTION TRANSFORMS INSTANCES, AND THE DIRECTION IS EVERYTHING

A mapping reduction from A to B is a computable function f such that x is in A if and only if $f(x)$ is in B. Written $A \leq B$, and read 'A reduces to B' or 'B is at least as hard as A'.

If $A \leq B$ and B is decidable, then A is decidable — transform the input with f and ask B's decider. **So contrapositively, if A is undecidable then B is undecidable.**

So to prove B undecidable, reduce a known undecidable problem TO it. $A \leq B$, with A already known to be hard. Not the other way round.

And that direction is where almost every attempt fails. Reducing B to HALT proves nothing about B — it establishes that B is no *harder* than halting, which is something you already suspected and is not what you set out to show. **State explicitly, in writing, what reduces to what, every time;** it is the single most effective guard against the error.

TO PROVE B UNDECIDABLE:

Assume a decider R for B.
Build a decider for HALT (or A_TM) using R.
Conclude R cannot exist.

THE STANDARD SHAPE: given an instance (M, w) of halting,
construct a NEW machine M' whose B-property holds exactly
when M halts on w.

EXAMPLE -- "does M accept the empty string?" is undecidable.

Given (M, w), build M':
M'(x):
 ignore x
 run M on w
 if that halts, accept

Then M' accepts the empty string
 $\Leftrightarrow$ M' accepts every string
 $\Leftrightarrow$ M halts on w

So a decider for "accepts the empty string" would
decide HALT. Contradiction.

NOTE WHAT M' DOES: it IGNORES ITS OWN INPUT and runs the
machine you are asking about. That one move -- build a
machine whose behaviour is CONSTANT and determined entirely
by whether M halts -- generates most of the reductions in
this subject. Learn it once.

2 Rice's theorem

RICE: EVERY NON-TRIVIAL SEMANTIC PROPERTY OF PROGRAMS IS UNDECIDABLE

Let P be a property of the *language* that a machine recognises — a property of its behaviour, not of its text. If P is **non-trivial**, meaning some machines have it and some do not, then P is undecidable. One theorem, proved by the construction of §1.

So: does this program compute the identity function? Does it ever output 7? Is it equivalent to that other program? Does it halt on all inputs? Is this branch dead code? Can this variable ever be null? All undecidable, in one stroke, without needing a separate reduction for each.

The conditions matter and are frequently misquoted. 'Semantic' excludes syntactic properties — 'has fewer than 100 states' or 'contains a goto' is decidable by inspection. 'Non-trivial' excludes the properties true of every machine or of none, which are trivially decidable by answering yes or no.

And that is why static analysis is hard in principle rather than in practice (Module 12). Rice's theorem says you are not failing to find the clever algorithm; there is no clever algorithm. Which is a genuinely useful thing to know before spending a month looking for one.

Property of a program	Semantic or syntactic?	Status
'Halts on every input'	Semantic	Undecidable
'Accepts only finitely many strings'	Semantic	Undecidable
'Computes the same function as this other program'	Semantic	Undecidable
'Contains a while loop'	Syntactic	Decidable — look at the source.
'Has at most 50 states'	Syntactic	Decidable
'Halts within 1000 steps'	Neither, strictly — it is a bounded behavioural property.	Decidable — simulate for 1000 steps and look.

The last row is the engineering escape, and it is the most important line in the table. Bound the resource and the question becomes decidable — 'halts within 1000 steps', 'uses at most 1 MB', 'no bug within 20 loop iterations'. Which is exactly what every practical analyser and every bounded model checker does (CSCE 625 Module 08 §3), and it is Module 12 §2's second engineering response. The unbounded quantifier is what costs the decidability, every time.

3 Undecidability far from programs

- **The Post correspondence problem.** Given a finite set of domino pairs, each with a string on top and a string on the bottom, is there a sequence (with repeats allowed) whose concatenated tops equal its concatenated bottoms? **Pure string manipulation, no reference to computation, and undecidable.**
- **Grammar ambiguity** (Module 03 §2), **and whether two context-free grammars generate the same language** — both of which have immediate consequences for parser tooling.
- **Tiling the plane** with a given finite set of tiles whose edges must match. **Geometry, and undecidable** — and the proof is connected to the existence of aperiodic tilings, which is why that question was interesting to logicians before it was interesting to crystallographers.
- **Hilbert's tenth problem:** does a given polynomial equation with integer coefficients have an integer solution? **Number theory, and undecidable** (Module 01 §1) — by a reduction that took decades to construct.
- **And matrix mortality:** given a finite set of integer matrices, can some product of them (with repeats) be the zero matrix? **Linear algebra, and undecidable. So undecidability is not confined to questions about programs** — these problems make no reference to computation at all, **and the reductions are how computation gets into them.**

POST CORRESPONDENCE IS THE BRIDGE

PCP is undecidable, by a reduction from halting that encodes a machine's computation history as a sequence of dominoes — the top string runs one step behind the bottom, so a matching sequence is exactly a valid halting computation.

And PCP is then easy to reduce to other string and grammar problems, because it is already in the right shape — which is why it is the intermediate step in so many proofs.

So grammar ambiguity's undecidability goes via PCP rather than directly from halting: build two grammars from a PCP instance such that they share a string exactly when the instance has a match. **The reduction is far more natural that way**, and attempting it directly from halting is painful.

Which is the general pattern worth taking away: **build a small library of hard problems in different shapes, and reduce from whichever one is structurally closest to your target** — exactly as CSCE 629 does with 3-SAT for logic, vertex cover for graphs, and Hamiltonian cycle for paths. The library is the tool, not any single problem in it.

4 The parallel with NP-hardness

	This course	CSCE 629 and CSCE 637
Reduce from	HALT or A_{TM} , or PCP as an intermediate.	3-SAT, or another known NP-complete problem.

	This course	CSCE 629 and CSCE 637
The function f must be	Computable.	Computable in polynomial time — the only difference.
The conclusion	No algorithm exists at all.	No polynomial-time algorithm exists, unless $P = NP$.
Status of the conclusion	Proved, unconditionally.	Conditional on $P \neq NP$, which is unproved.
Direction	Known hard problem $\rightarrow$ your problem.	Known hard problem $\rightarrow$ your problem.

The technique is identical and only the resource bound on f differs — which is why learning reductions here transfers directly to complexity, and why the direction error is the same error in both subjects. It is also why the two courses are in the same semester.

THE PRACTICAL UPSHOT

When you meet a problem that resists solution, try to prove it impossible. An hour spent attempting a reduction is far cheaper than a month of algorithm design, and the attempt frequently clarifies the problem even when it fails.

And a successful reduction redirects the effort productively — toward restricting the input, accepting an approximation, or returning a partial answer (Module 12) — all of which are real engineering options rather than consolation prizes.

Rice's theorem makes this unusually fast for program analysis: if the property you want to decide is semantic and non-trivial, stop immediately and start designing the approximation. No reduction needs to be constructed; the theorem has done it.

Which is the professional value of this module. Recognising impossibility early is a skill, and it saves more time than any optimisation — and it is the one skill in this course that transfers to every other.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — chapter 5	Mapping reductions, Rice's theorem, and the Post correspondence problem — the reference for this module, including the PCP reduction in full.
MIT 18.404J — lectures 10–12 (free)	The reductions worked at the board, which is the right way to see them the first time.
Rice — Classes of recursively enumerable sets (free via JSTOR listing)	§2's theorem, in the original. Short.

Resource	What it covers for this module
Berger — The undecidability of the domino problem; and the Wang tiles literature	§3's tiling result , and its connection to aperiodic tilings is worth following.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write out the definition of a mapping reduction** and state the direction rule in your own words.
- 2 **Construct the 'accepts the empty string' reduction** yourself without looking.
- 3 **Prove three more properties undecidable** by reduction: 'accepts a finite language', 'halts on all inputs', 'is equivalent to the always-reject machine'.
- 4 **Deliberately do one reduction backwards** and explain precisely what it proves and does not.
- 5 **Apply Rice's theorem to five properties** and verify the non-triviality condition each time.
- 6 **Find a property that Rice does not cover** and say why.
- 7 **Bound a property** — 'halts within n steps' — and write the decider.
- 8 **Solve three small PCP instances by hand** and one that has no solution.
- 9 **Sketch the PCP-to-grammar-ambiguity reduction.**
- 10 **Take an unsolved problem from your own work** and spend an hour trying to prove it undecidable. Report what you learned either way.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define a mapping reduction and state what it preserves.
- 2 Which direction proves B undecidable, and what does the other direction prove?
- 3 Give the workhorse construction and explain the ignore-the-input move.
- 4 State Rice's theorem with both conditions.
- 5 Classify six properties as semantic, syntactic, or bounded.
- 6 What does bounding a resource buy?
- 7 Name five undecidable problems that are not about programs.
- 8 Why is PCP the bridge?
- 9 Compare undecidability and NP-hardness reductions on five axes.
- 10 What is the practical upshot of this module?

